

Tabellen

August 24, 2026

```
[22]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.constants as cc
from scipy.stats import norm
from scipy.optimize import curve_fit

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab

# Zum Auslesen von Dateien und ähnlichem
from pathlib import Path

from pytexit import py2tex
import re

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap
```

```
# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002
```

```
[23]: base = Path.cwd()
Ausgabe_path = Path(base/"Diagramme")
print(Ausgabe_path)
```

c:\Users\samue\OneDrive\Dokumente\FP01\Diagramme

Delete UTF8 False encoding

```
[24]: # def deletevocal(text):
#     text=textwrap.dedent(text)
#     patterna = r'[][a] '
#     text=re.sub(patterna,"ä",text)
#     patternu=r'[][u] '
#     text=re.sub(patternu,"ü",text)
#     patterno=r'[][o] '
#     text=re.sub(patterno,"ö",text)
#     return text

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevocal(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
```

```

        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
        ↪ wie es ist
        return umlaute.get(buchstabe, buchstabe)
        # Das Pattern sucht nach " gefolgt von einem beliebigen Buchstaben aus der
        ↪ Auswahl, [X] findet genau ein Zeichen aus Menge X
        return re.sub(r'"([auoAUO])', convert, text)

```

```

[25]: print(deletevocal("""
"""))

```

Runden signifikanter Stellen

```

[26]: def round_to_sigs(val,errVal=None,einheiten=None):
        """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↪ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↪ Daher musste hier mit dezimal gearbeitet werden.
        Zudem sollten besonders kleine und große Messwerte in der
        ↪ Dezimalschreibweise geschrieben werden, damit diese auch
        für Protokolle geeignet sind.

        Parameter
        -----
        **val** : float
            Messwert

        **errVal** : float
            Ungenauigkeit des Messwertes

        Return
        -----
        **value_rounded** : str
            Gerundeter Messwert

        **error_rounded** : str
            Gerundete Ungenauigkeit des Messwertes

        **res** : str
            "Messwert \|pm Fehler"
        """
        einheiten_str = einheiten if einheiten is not None else ""
        if errVal is not None:
            # Daten zu Dezimal wechseln, da Floats probleme machen
            val = Decimal(str(val))

```

```

errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal))))          # Die erste
↳signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3:    # wenn
↳1,2,3 erste signifikante Stelle, dann kommt eine zweite
↳Nachkommastellenposition hinzu
        exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'),
↳rounding=ROUND_HALF_UP) * scale          # mit scale wird das Komma so
↳verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann
↳alle Nachkommastellen weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'),
↳rounding=ROUND_CEILING) * scale

num = f"\num{{{val_round}}({err_round})}"
qty = f"\qty{{{val_round}}({err_round})}{{{einheiten_str}}}" if
↳einheiten else num

    return float(val_round), float(err_round), num, qty
else:
    val = Decimal(str(val))
    exp = int(math.floor(math.log10(float(val))))
    if round(float(val / (Decimal(10) ** exp))) < 3:      # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
        exp -= 1
        scale = Decimal(10) ** exp
        val_round = (val / scale).quantize(Decimal('1'),
↳rounding=ROUND_HALF_UP) * scale
        num = f"\num{{{val_round}}}"
        qty = f"\qty{{{val_round}}}{{{einheiten_str}}}" if einheiten else num
    return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

    # wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10~e Prefixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```
[27]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamaters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable) # Die Funktion
    ↪wird nach der fehlerbehafteten Variable abgeleitet
        term=sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term)) # Fehler werden
    ↪quadratisch aufsummiert
        errProneParamaters.append((variable,Delta))

    absolute_err=sp.simplify(sp.sqrt(error),rational = True)
    relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)
```

```

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[28]: def sigma_abweichung(p1, err_p1,p2,err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
    ↪Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

    if abweichung == 0:
        return 0.0, "\\num{0}"

    exp = int(math.floor(math.log10(float(abweichung))))
    if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪Nachkommastellenposition hinzu
        exp -= 1
        scale = Decimal(10) ** exp
        abweichung_round = (abweichung / scale).quantize(Decimal('1'),
        ↪rounding=ROUND_CEILING) * scale
        res = f"\\num{{{abweichung_round}}}"

    return float(abweichung_round),res

```

```

[29]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2

```

```

#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(namM_list,nam1, nam2, einheiten, val1_array,
    ↪err1_array, val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        \\text{{Messreihe}} & {nam1}[\\unit{{{{einheiten}}}}] &
    ↪{nam2}[\\unit{{{{einheiten}}}}] & \\text{{abs. Abw.}}[\\unit{{{{einheiten}}}}] &
    ↪\\text{{proz. Abw.}}[\\unit{{{{percent}}}}] & S[\\sigma] \\\\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for namM, val1, err1, val2, err2 in zip(namM_list, val1_array, err1_array,
    ↪val2_array, err2_array):

        VAL1=round_to_sigs(val1,err1,r'[2]
        if err2!=0:
            VAL2=round_to_sigs(val2,err2,r'[2]
        else:
            VAL2=val2

        abs=np.abs(val1-val2)
        abs_delta=np.sqrt(err1**2+err2**2)
        if abs_delta!=0:
            SIGMA=sigma_abweichung(val1,err1,val2,err2)[0]
        else:
            SIGMA=0.0

        rel=abs/np.abs(val1)*100
        if abs != 0:
            rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
            ABS = round_to_sigs(abs, abs_delta, r'[2]
            REL = round_to_sigs(rel, rel_delta, r'[2]
        else:
            rel_delta = 0
            ABS = "0"
            REL = "0"

```

```

# Eine saubere Zeile nur mit den Werten für LaTeX
row = f"          {namM} & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
table_rows.append(row)

body_str = "\\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{nam1}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[30]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
#     else:
#         SIGMA=0.0
#     ABS=round_to_sigs(abs,abs_delta,r'')[2]
#     rel=abs/np.abs(val2)*100
#     rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else
↪ 0
#     REL=round_to_sigs(rel,rel_delta,r'')[2]
#     output = textwrap.dedent(f"""
#         \\begin{{table}}[htb]
#         \\centering
#         \\caption{{}}
#         \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
#         \\toprule
#         ↪
↪ {nam1}[\\unit{{einheiten}}]&{nam2}[\\unit{{einheiten}}]&\\text{{abs. Abw.
↪ }}[\\unit{{einheiten}}]&\\text{{proz. Abw.
↪ }}[\\unit{{percent}}]&S[\\sigma]\\\\\\midrule
#         {VAL1}&\\num{{val2}}&{ABS}&{REL}&{SIGMA}\\\\\\
#         \\bottomrule
#         \\end{{tabularx}}
#         \\label{{table:Vergleich_{nam1}}}

```



```

#         \\end{{table}}
#         """)
#         print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r'[2]
    else:
        VAL2=val2
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r'[2]
        REL = round_to_sigs(rel, rel_delta, r'[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    output = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZZx}}
        \\toprule

```

```

        \\\unit{{{einheiten}}}]&{{nam2}}[\\unit{{{einheiten}}}]&\\text{{abs. Abw.
        \\unit{{{einheiten}}}]&\\text{{proz. Abw.
        \\unit{{{percent}}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{{nam1}}}}
        \\end{{table}}
    """)
    print(output)

```

```
[31]: round_to_sigs(66.50,1.5,r'')
```

```
[31]: (66.5, 1.5, '\\num{66.5(1.5)}', '\\num{66.5(1.5)}')
```

1 Emitter

```

[32]: # Liste der Namen-Strings
names = [
    "R_C",
    r"R_E",
    r"R_1",
    r"R_2",
    r"U_{\text{CE}}",
    r"U_{\text{BE}}",
    r"U_{\text{C}}",
    r"I_{\text{C}}",
    r"U_{\text{E}}",
    r"U_{R_1}",
    r"U_{R_2}",
]

# Werte mit E12-Normreihe
values_norm = np.array(
    [
        6800.0, # R_C in Ohm (6.8 kOhm)
        330.0, # R_E in Ohm
        820000.0, # R_1 in Ohm (820 kOhm)
        68000.0, # R_2 in Ohm (68 kOhm)
        7.87, # U_CE in V
        0.60, # U_BE in V
        6.80, # U_C in V
        0.001, # I_C in A
        0.33, # U_E in V
        14.07, # U_R1 in V (15 V - 0.93 V)
        0.93, # U_R2 in V
    ]
)

```

```

    ]
)
Delta_norm=np.full_like(values_norm,0)

values_R=np.array([6810.0,324.7,824000.0,66500.0])
Delta_R=values_R*0.002+np.array([1,0.1,100,10])

values_U=np.array([6.82,0.622,7.78,0.00114,0.37,13.93,0.991])
Delta_U=values_U*np.array([0.0005,0.0005,0.0005,0,0.0005,0.0005,0.0005])+np.
    ↪array([0.01,0.001,0.01,5*1e-6,0.01,0.01,0.001])
values=np.concatenate((values_R,values_U))
Delta_values=np.concatenate((Delta_R,Delta_U))

```

[33]: `compare_table_array(names,r'x_{\text{exp}}',r'x_{\text{ber}}',r'',values,Delta_values,values_r`

```

\begin{table}[htbp]
\centering
\caption{Vergleich von  $x_{\text{exp}}$  und  $x_{\text{ber}}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZZ}
\toprule
\text{Messreihe} &  $x_{\text{exp}}$  [\unit{}]&  $x_{\text{ber}}$  [\unit{}]& \text{abs. Abw.} [\unit{}]& \text{proz. Abw.} [\unit{\percent}]& S[\sigma] \\
\\ \midrule
R_C & \num{6810(15)} & 6800.0 & \num{10(15)} & \num{0.15(0.22)} & 0.7 \\
R_E & \num{324.7(0.8)} & 330.0 & \num{5.3(0.8)} & \num{1.63(0.24)} & 8.0 \\
\\
R_1 & \num{824000(1800)} & 820000.0 & \num{4000(1800)} & \num{0.49(0.22)} & \\
& & & & & 2.3 \\
R_2 & \num{66500(150)} & 68000.0 & \num{1500(150)} & \num{2.26(0.22)} & \\
& & & & & 11.0 \\
U_{\text{CE}} & \num{6.820(0.014)} & 7.87 & \num{1.050(0.014)} & & \\
& & & & & \num{15.40(0.20)} & 80.0 \\
U_{\text{BE}} & \num{0.6220(0.0014)} & 0.6 & \num{0.0220(0.0014)} & & \\
& & & & & \num{3.54(0.22)} & 17.0 \\
U_{\text{C}} & \num{7.780(0.014)} & 6.8 & \num{0.980(0.014)} & & \\
& & & & & \num{12.60(0.18)} & 80.0 \\
I_{\text{C}} & \num{0.001140(0.000005)} & 0.001 & & & \\
& & & & & \num{0.000140(0.000005)} & \num{12.3(0.5)} & 30.0 \\
U_{\text{E}} & \num{0.370(0.011)} & 0.33 & \num{0.040(0.011)} & & \\
& & & & & \num{11(3)} & 4.0 \\
U_{R_1} & \num{13.930(0.017)} & 14.07 & \num{0.140(0.017)} & & \\
& & & & & \num{1.01(0.13)} & 9.0 \\
U_{R_2} & \num{0.9910(0.0015)} & 0.93 & \num{0.0610(0.0015)} & & \\
& & & & & \num{6.16(0.16)} & 50.0 \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_x_{\text{exp}}}
\end{table}

```

1.1 Verstärkung

```
[34]: # bei 5kHz
U_E=0.486
Delta_U_E=0.00278
U_A=9.43
Delta_U_A=0.0287
V=U_A/U_E
Delta_V=V*np.sqrt((Delta_U_E/U_E)**2+(Delta_U_A/U_A)**2)
V,Delta_V,_,latexV=round_to_sigs(V,Delta_V,r'')
print(latexV)
```

$\text{\num{19.40(0.13)}}$

1.2 Grenzfrequenzvergleich

```
[35]: f_ug_exp=31.2
Delta_f_ug_exp=2.5
f_theo=37.40
compare_table(r'f_{\text{ug}}^{\text{exp}}',r'f_{\text{ug}}^{\text{exp}}',r'\hertz',f_ug_exp,D
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $f_{\text{ug}}^{\text{exp}}$  und  $f_{\text{ug}}^{\text{exp}}$ }
\begin{tabularx}{\textwidth}{ZZZx}
\toprule
f_{\text{ug}}^{\text{exp}}[\text{hertz}]&f_{\text{ug}}^{\text{exp}}[\text{hertz}]&\text{abs. Abw.}[\text{hertz}]&\text{proz. Abw.}[\text{percent}]&S[\sigma]\\\midrule
\text{\num{31.2(2.5)}}&\text{\num{37.4}}&\text{\num{6.2(2.5)}}&\text{\num{20(9)}}&\text{\num{2.5}}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_f_{\text{ug}}^{\text{exp}}}
\end{table}
```

1.3 Ein Aus Widerstand

```
[36]: R_exp=np.array([42.53,6.91])
Delta_R_exp=R_exp*0.05
R_calc=np.array([42.55,6.8])
Delta_R_calc=np.array([0,0])
names=[r'R_{\text{EIN}}',r'R_{\text{AUS}}']
compare_table_array(names,r'R_{\text{exp}}',r'R_{\text{calc.}}',r'\kilo\ohm',R_exp,Delta_R_exp,R_calc,Delta_R_calc)
```

```
\begin{table}[htbp]
```

```

\centering
\caption{Vergleich von  $R_{\text{exp}}$  und  $R_{\text{calc.}}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZZ}
\toprule
\text{Messreihe} &  $R_{\text{exp}}$  [\unit{kilo\ohm}] & & & & \\
 $R_{\text{calc.}}$  [\unit{kilo\ohm}] & \text{abs. Abw.} [\unit{kilo\ohm}] & & & & \\
\text{proz. Abw.} [\unit{percent}] &  $S[\sigma]$  & \midrule
 $R_{\text{EIN}}$  &  $\text{num}\{42.5(2.2)\}$  & 42.55 &  $\text{num}\{0.0(2.2)\}$  &  $\text{num}\{0(6)\}$  & \\
0.01 & & & & & \\
 $R_{\text{AUS}}$  &  $\text{num}\{6.9(0.4)\}$  & 6.8 &  $\text{num}\{0.1(0.4)\}$  &  $\text{num}\{2(6)\}$  & 0.4 \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_ $R_{\text{exp}}$ }
\end{table}

```

1.4 Stromstärken Berechnung

```

[37]: U=np.array([7.78,7.7])
Delta_U=U*0.0005+0.01
R=np.array([6800,471.9])
Delta_R=R*0.002+np.array([10,0.1])
I=U/R
Delta_I=I*np.sqrt(Delta_R**2/R**2 + Delta_U**2/U**2)
I,Delta_I,_,_=v_round(I,Delta_I,r'\milli\ampere')
print(I)
print(Delta_I)

```

```

[0.001144 0.01632 ]
[5.e-06 5.e-05]

```

```

[38]: gff("U/R", "U,R")

```

Funktion:

$$\frac{U}{R}$$

$$\frac{U}{R}$$

Absoluter Fehler:

$$\sqrt{\frac{1}{R^4} (\Delta_R^2 U^2 + \Delta_U^2 R^2)}$$

$$\sqrt{\frac{\Delta_R^2}{R^4} U^2 + \frac{\Delta_U^2}{R^2}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_U^2}{U^2}}$$

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_U^2}{U^2}}$$

```
[38]: (U/R,
      sqrt((Delta_R**2*U**2 + Delta_U**2*R**2)/R**4),
      sqrt(Delta_R**2/R**2 + Delta_U**2/U**2),
      [(U, Delta_U), (R, Delta_R)])
```

```
[49]: names = [r'V',r'\Phi\;[\unit{\degree}']
values=np.array([0.99,2.0])
Delta_values=np.array([0.11,2])
values_theo=np.array([1,0])
compare_table_array(names,r'x_{\text{exp}}',r'x_{\text{ber}}',r'',values,Delta_values,values_theo,
                    array([0,0]))
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $x_{\text{exp}}$  und  $x_{\text{ber}}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZZ}
\toprule
\text{Messreihe} &  $x_{\text{exp}}$ [\unit{ }] &  $x_{\text{ber}}$ [\unit{ }] & \text{abs. Abw.}[\unit{ }] & \text{proz. Abw.}[\unit{\percent}] & S[\unit{\sigma}] \\
\midrule
V &  $\text{0.99(0.11)}$  & 1 &  $\text{0.01(0.11)}$  &  $\text{1(12)}$  & 0.1 \\
\Phi\;[\unit{\degree}] &  $\text{2.0(2.0)}$  & 0 &  $\text{2.0(2.0)}$  & &  $\text{100(150)}$  & 1.0 \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_x_{\text{exp}}}
\end{table}
```

2 Kollektorschaltung

```
[39]: names = [
    r"R_E",
    r"R_1",
    r"R_2",
    r"U_{\text{E}}",
    r"I_{\text{E}}",
    r"U_{\text{BE}}",
    r"U_{R_1}",
    r"U_{R_2}",
]

# Theoretische berechnete Werte (in SI-Einheiten: Ohm, Volt, Ampere)
values_theo = np.array(
    [
        500.00, # R_E in Ohm
        30666.67, # R_1 in Ohm (30.67 kOhm)
        43200.00, # R_2 in Ohm (43.20 kOhm)
```

```

        7.50, # U_E in V
        0.015, # I_E in A (Vorgegebener Wert: 15 mA)
        0.60, # U_BE in V (Vorgegebener Wert)
        6.90, # U_R1 in V (U_V - U_R2)
        8.10, # U_R2 in V
    ]
)

# Werte mit E12-Normreihe
values_norm = np.array(
    [
        470.00, # R_E in Ohm
        27000.00, # R_1 in Ohm (27 kOhm)
        39000.00, # R_2 in Ohm (39 kOhm)
        7.05, # U_E in V (470 Ohm * 15 mA)
        0.015, # I_E in A
        0.60, # U_BE in V
        6.90, # U_R1 in V
        8.10, # U_R2 in V
    ]
)

Delta_norm=np.full_like(values_norm,0)

values_R=np.array([471.9,26940.0,38400.0])
Delta_R=values_R*0.002+np.array([0.1,10,10])

values_U=np.array([7.70,0.01632,0.679,6.59,8.38])
Delta_U=values_U*np.array([0.0005,0,0.0005,0.0005,0.0005])+np.array([0.
    ↪01,5*1e-5,0.001,0.01,0.01])
values=np.concatenate((values_R,values_U))
Delta_values=np.concatenate((Delta_R,Delta_U))

```

[40]: `compare_table_array(names,r'x_{\text{exp}}',r'x_{\text{ber}}',r'',values,Delta_values,values_r`

```

\begin{table}[htbp]
\centering
\caption{Vergleich von  $x_{\text{exp}}$  und  $x_{\text{ber}}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZZ}
\toprule
\text{Messreihe} &  $x_{\text{exp}}$  [\unit{}]&  $x_{\text{ber}}$  [\unit{}]& \text{abs. Abw.} [\unit{}]& \text{proz. Abw.} [\unit{\percent}]& S[\sigma]
\\ \midrule
R_E & \num{471.9(1.1)} & 470.0 & \num{1.9(1.1)} & \num{0.40(0.23)} & 1.9
\\
R_1 & \num{26940(70)} & 27000.0 & \num{60(70)} & \num{0.22(0.24)} & 1.0
\\
R_2 & \num{38400(90)} & 39000.0 & \num{600(90)} & \num{1.56(0.23)} & 7.0
\\

```

```

U_{\text{E}} & \num{7.700(0.014)} & 7.05 & \num{0.650(0.014)} &
\num{8.44(0.19)} & 50.0 \\
I_{\text{E}} & \num{0.01632(0.00005)} & 0.015 & \num{0.00132(0.00005)} &
\num{8.1(0.4)} & 30.0 \\
U_{\text{BE}} & \num{0.6790(0.0014)} & 0.6 & \num{0.0790(0.0014)} &
\num{11.63(0.20)} & 60.0 \\
U_{R_1} & \num{6.590(0.014)} & 6.9 & \num{0.310(0.014)} &
\num{4.70(0.21)} & 24.0 \\
U_{R_2} & \num{8.380(0.015)} & 8.1 & \num{0.280(0.015)} &
\num{3.34(0.17)} & 20.0 \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_x_{\text{exp}}}
\end{table}

```

2.1 Verstärkung

```

[46]: # bei 5kHz
U_E=755
Delta_U_E=9.82
U_A=746
Delta_U_A=79.5
V=U_A/U_E
Delta_V=V*np.sqrt((Delta_U_E/U_E)**2+(Delta_U_A/U_A)**2)
V,Delta_V,_,latexV=round_to_sigs(V,Delta_V,r'')
print(latexV)

```

```

\num{0.99(0.11)}

```

2.2 Grenzfrequenz

```

[41]: f_ug_exp=30.5
Delta_f_ug_exp=2.5
f_theo=49.10
compare_table(r'f_{\text{ug}}^{\text{exp}}',r'f_{\text{ug}}^{\text{exp}}',r'\hertz',f_ug_exp,D

```

```

\begin{table}[htbp]
\centering
\caption{Vergleich von  $f_{\text{ug}}^{\text{exp}}$  und  $f_{\text{ug}}^{\text{exp}}$ }
\begin{tabularx}{\textwidth}{ZZZx}
\toprule
f_{\text{ug}}^{\text{exp}}[\unit{\hertz}]&f_{\text{ug}}^{\text{exp}}[\unit{\hertz}]&\text{abs. Abw.}[\unit{\hertz}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\num{30.5(2.5)}&49.1&\num{18.6(2.5)}&\num{61(10)}&\num{8}\\\bottomrule

```



```

\end{tabularx}
\label{table:Vergleich_f_{\text{ug}}^{\text{exp}}}
\end{table}

```

```

[43]: names = [r'V',r'\Phi\;[\unit{\degree}]]'
values=np.array([19.4,177.0])
Delta_values=np.array([0.13,8])
values_theo=np.array([20,180])
compare_table_array(names,r'x_{\text{exp}}',r'x_{\text{ber}}',r'',values,Delta_values,values_t
↪array([0,0]))

```

```

\begin{table}[htbp]
\centering
\caption{Vergleich von  $x_{\text{exp}}$  und  $x_{\text{ber}}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZZ}
\toprule
\text{Messreihe} & x_{\text{exp}}[\unit{ }] & x_{\text{ber}}[\unit{ }] & \text{abs. Abw.}[\unit{ }] & \text{proz. Abw.}[\unit{\percent}] & S[\unit{\sigma}] \\
\\ \midrule
V & \num{19.40(0.13)} & 20 & \num{0.60(0.13)} & \num{3.1(0.7)} & 5.0 \\
\Phi\;[\unit{\degree}] & \num{177(8)} & 180 & \num{3(8)} & \num{2(5)} & 0.4 \\
\\ \bottomrule
\end{tabularx}
\label{table:Vergleich_x_{\text{exp}}}
\end{table}

```